GOSIM

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris

GOSIM Paris 2026

Pie Overview Guide Reference Models Roadmap Community GitHub Search ⌘ K

A *programmable* LLM serving system.

High-performance inference engine where you write the loop.
Forward passes are library calls in your inferlet.

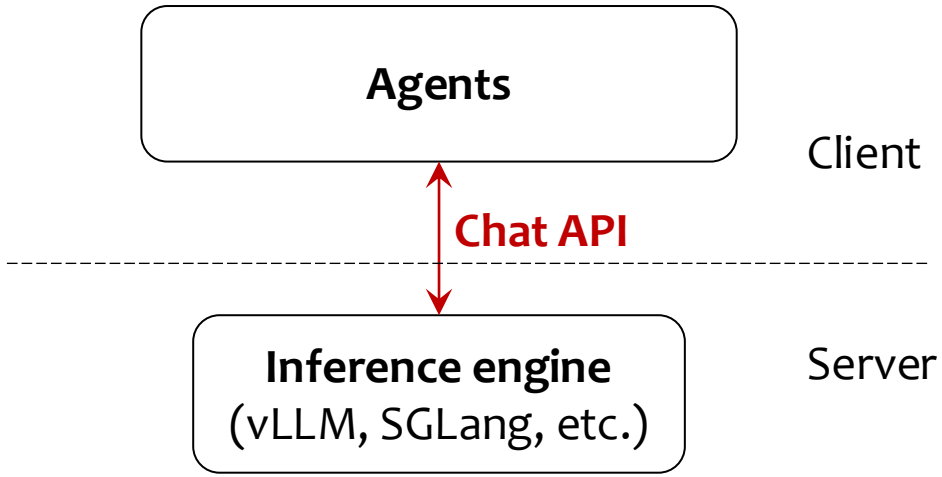
What is Pie? Get started GitHub

Lin Zhong, Ph.D., Yale University

https://pie-project.org

Serve programs, not prompts

Today's inference engines serve Prompts



LLM as locked up Oracle

Emerging agentic apps are programs

Have memory

Use tools

Branch, fork

Custom reasoning logic

- Graph-of-thought
- Beamsearch
- Constrained decoding
- …

“Prompt-serving” is ill-fit for agentic apps

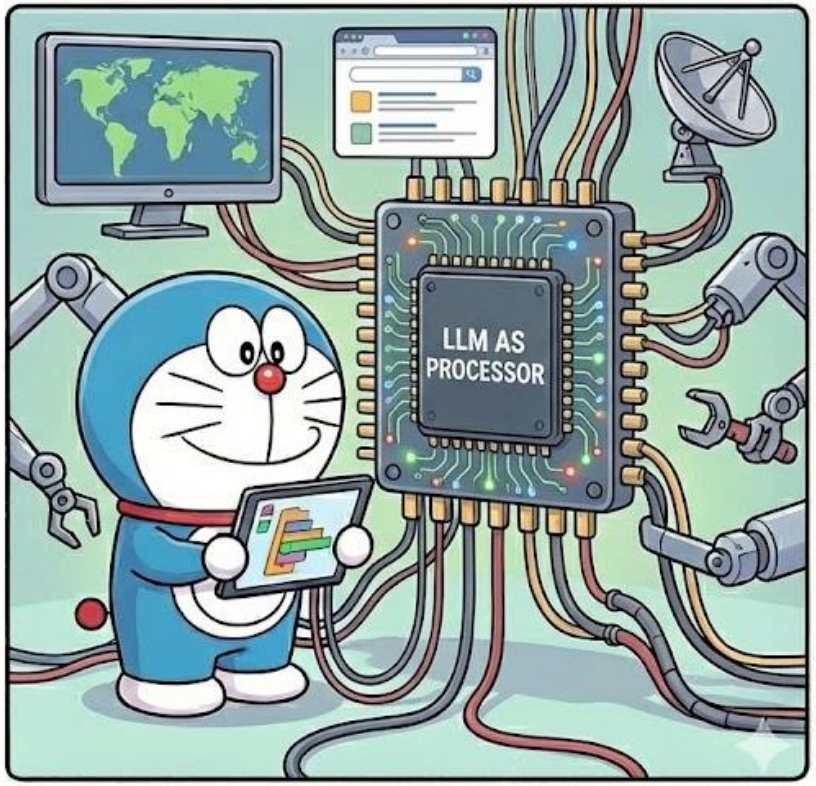
- KV cache is invisible to the app
- Tool calls are network round trips
- Custom decoding requires modifying the engine

More tokens ($), longer latency, and lost control

From serving “Prompt” to serving “Program”

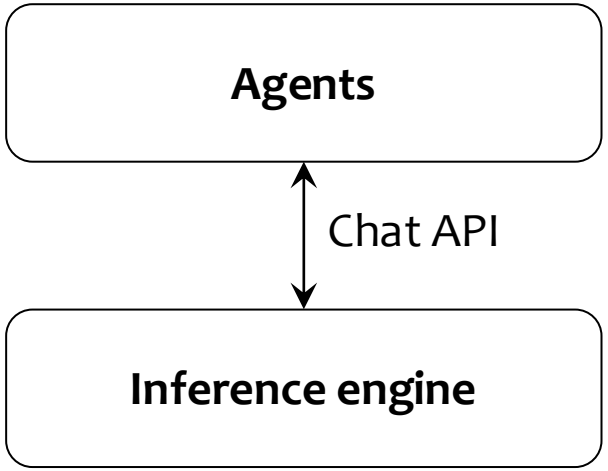
pie

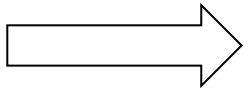
https://pie-project.org

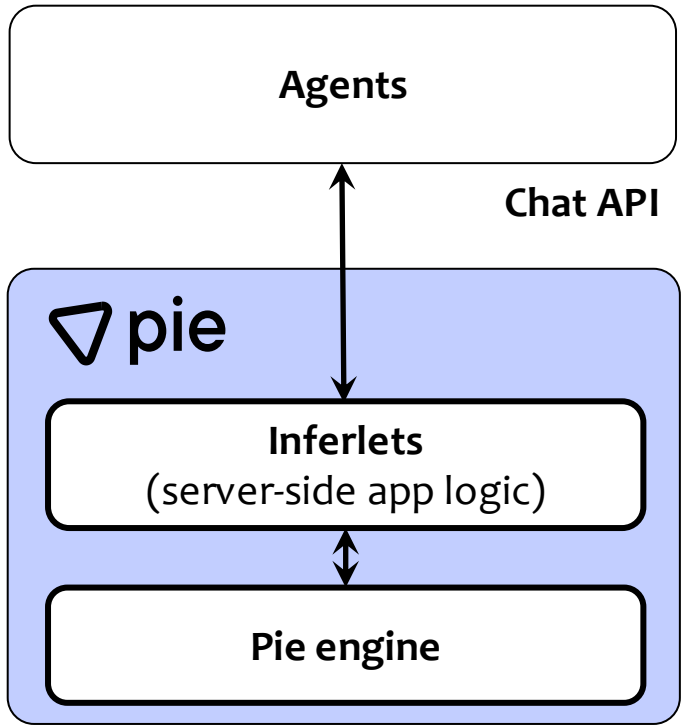


- Application-specific optimization
- Programmatic model control
- Less engineering

From serving “Prompt” to serving “Program”

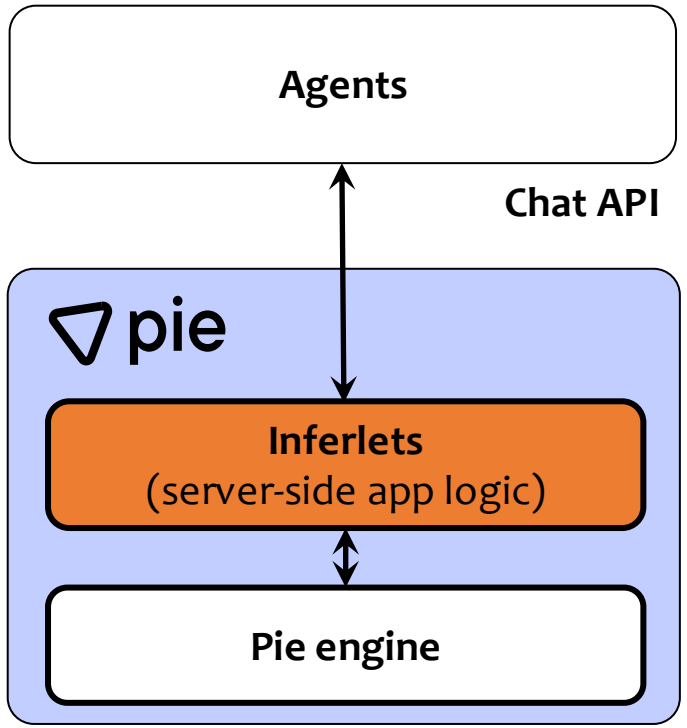






Inferlet: program that controls inference workflow

- Rust, Python, and TypeScript
- compiled and executed through WebAssembly
- Pie SDK: Library for writing inferlets
- Bakery: Inferlet toolchain



- Token-level forward pass control
  - Constrained decoding, Speculative Decoding, Custom Samplers and raw logits, custom attention mask, LoRA adapters
- Fine-grained KV cache control
  - Prompt prefix caching, Best-of-N and tree of thought, Sliding window and attention sink, Resumable sessions
- Integrated I/O
  - Function calling, MCP, virtual filesystem, embedded language runtime, HTTP servers

HEURISTIC If you've thought 'I wish vLLM exposed X', X is probably an inferlet in Pie.

Real inferlets that ship in the repo today

Each is a small Rust program — ideas you can clone, run, and extend in an afternoon.

agent-react · agent-codeact

ReAct and CodeACT loops with in-engine tool execution and JS code as actions.

agent-swarm

Multi-agent coordination via inferlet-to-inferlet pub/sub messaging.

constrained-decoding

Grammar- and JSON-schema-constrained sampling without engine patches.

text-completion-spec · cacheback · jacobi

Speculative decoders with custom drafters and verification strategies.

attention-sink · windowed-attention

Custom attention masks for bounded-memory long generation.

watermarking · raw-logits-demo

Pull full distributions and modify logits before sampling, per token.

knowledge-graph · graph-of-thought

Reasoning topologies expressed as forked, shared KV-cache contexts.

openresponses · mcp-example

OpenAI Responses-style API and MCP tool surfaces — built as inferlets.

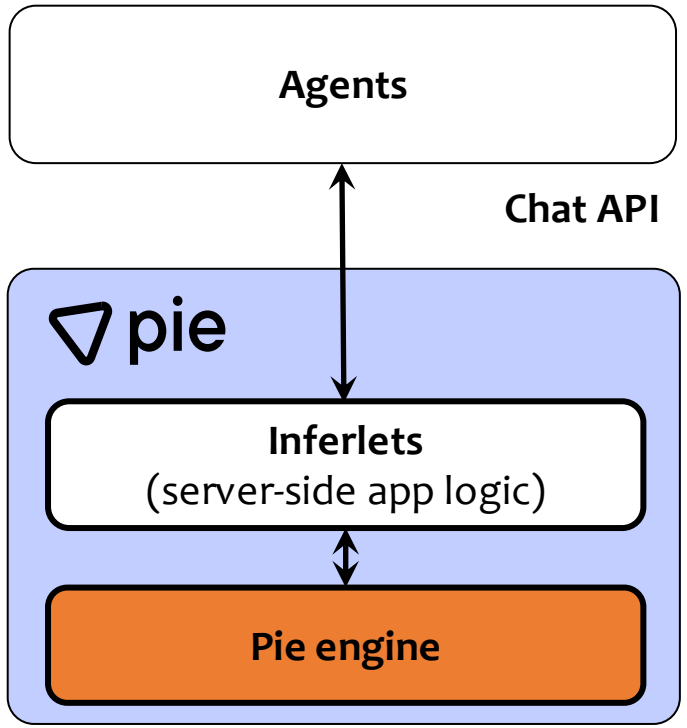
page-trim-bench · best-of-n

Throughput experiments and parallel sampling, written in user space.

https://pie-project.org/docs/guide/examples/overview

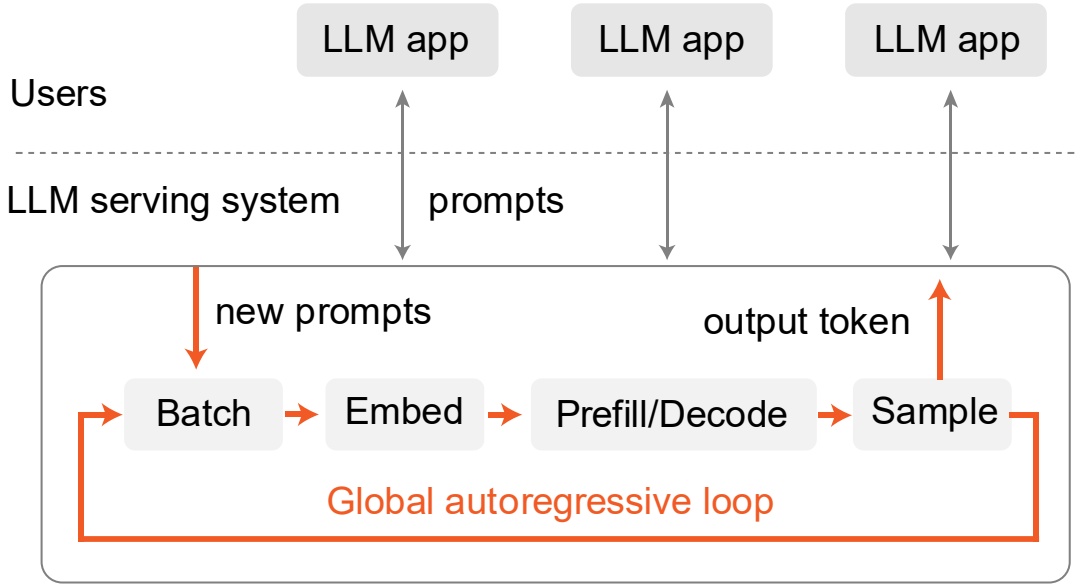
Pie engine: Operating System (OS) for inferlets

- Runtime (written in Rust)
  - Wasmtime-based WebAssembly runtime
  - Inferlet scheduler
  - KV cache mapping
  - Contention management
  - WebSocket protocol
- Drivers
  - A CUDA driver, and a local driver for CPU and Apple Silicon
  - Experimental support *vLLM* and *SGLang* as drivers



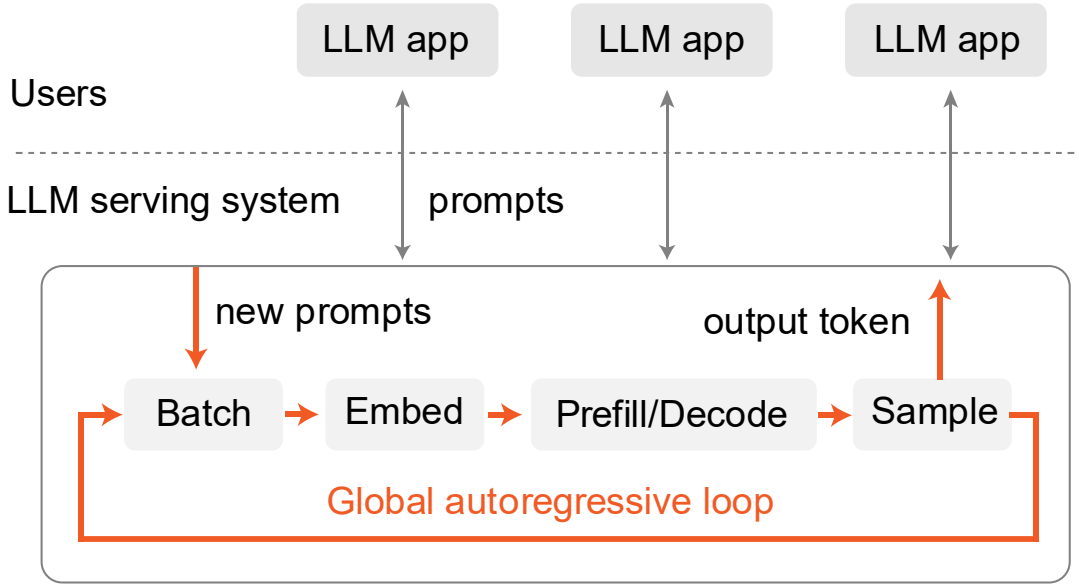
Current inference engines implement a monolithic loop

vLLM/SGLang

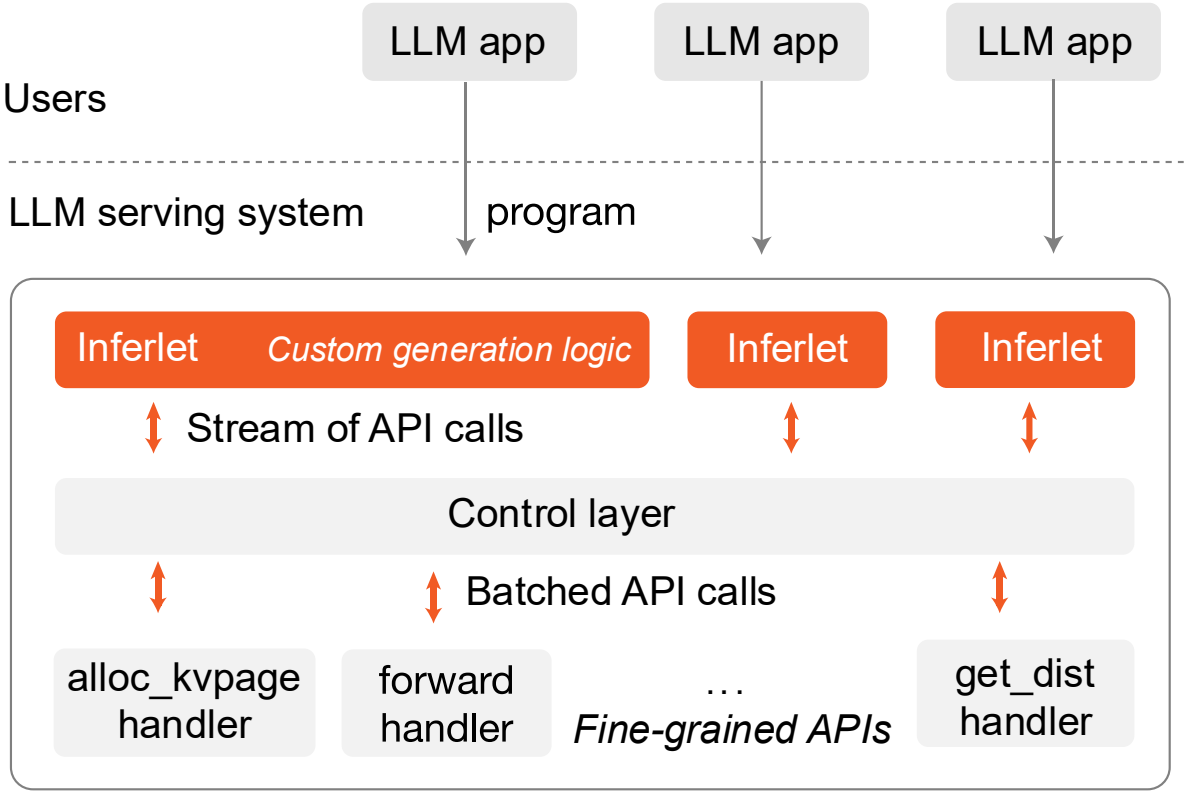


From monolithic loop to **disaggregated architecture**

vLLM/SGLang

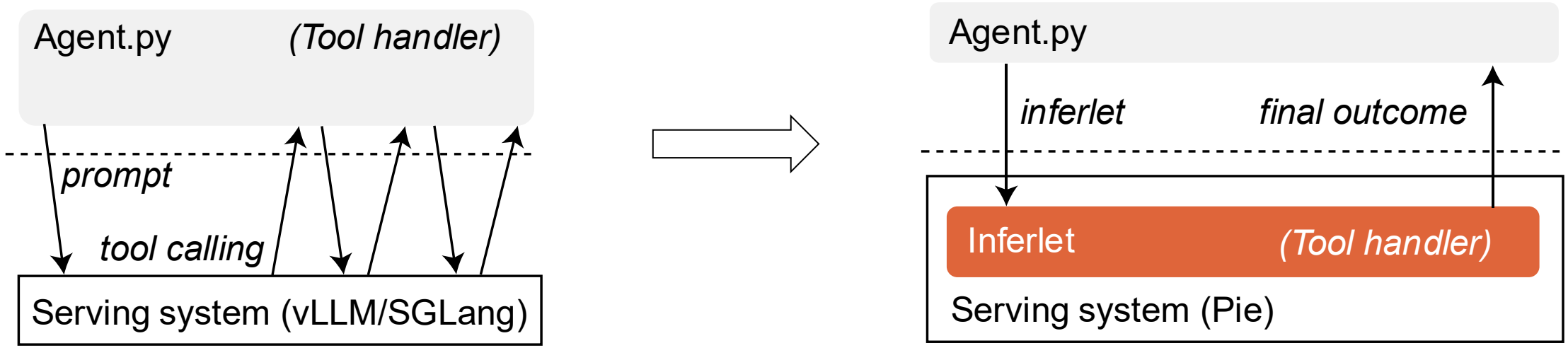


Pie: programmable serving



color denotes the control flow

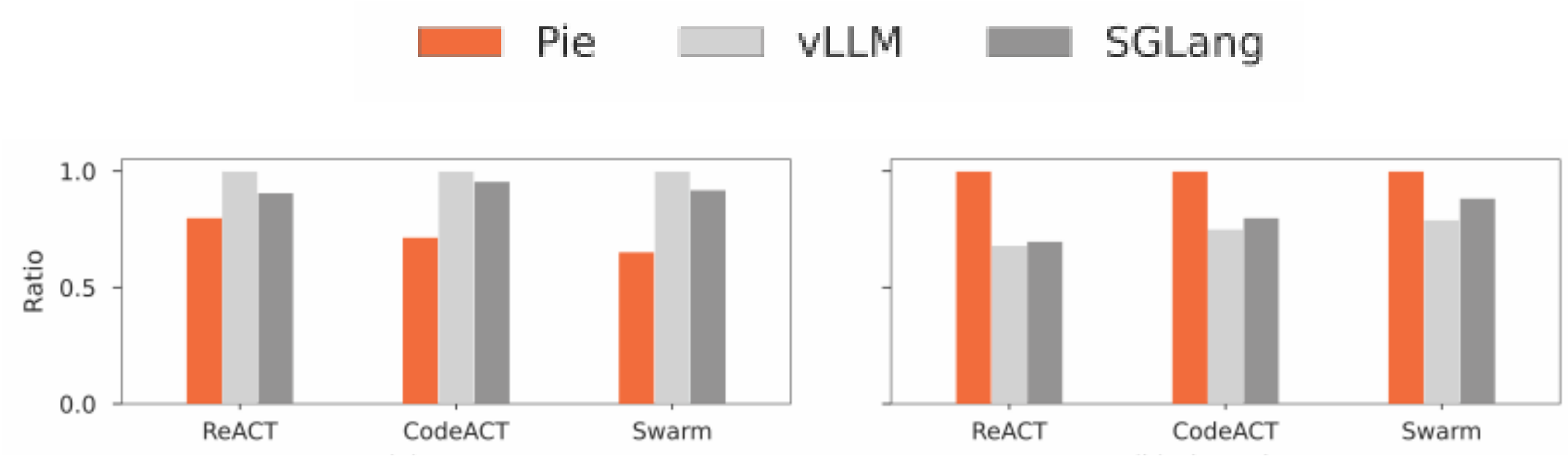
Pie allows in-server tool use



Lower latency, higher throughput, and fewer tokens

Inferlets incur fewer boundary crossings, and retain KV cache between calls

Pie accelerates agentic workflows



Lower Latency

Higher Throughput

Pie improves both latency and throughput by up to 1.5×.

Pie enables novel app-specific optimizations

App knowledge

1. Some tools used *more often* than others.
2. Some tools are *fire-and-forget*.
3. Some tools invoked *only once*.

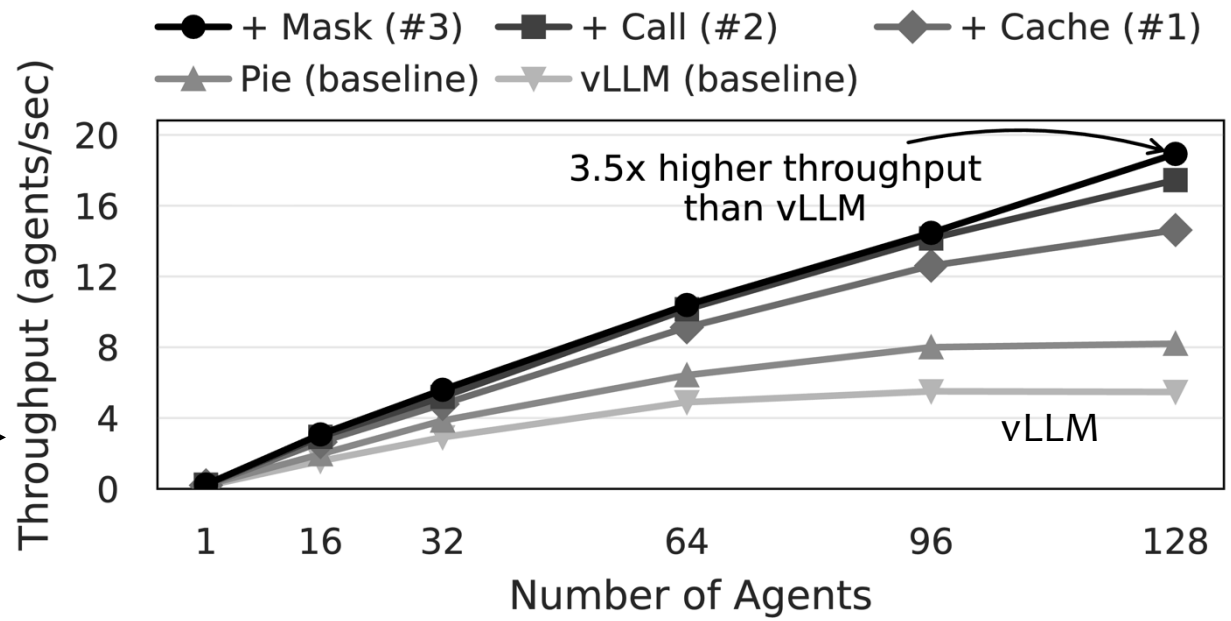
Optimization opportunities

#1. Retain the KV cache of frequently used tool

#2. Concurrently call a tool

#3. Drop the KV cache of used tools

Compounding optimizations



ingim@Workstation:~ ⌥⌘2

```
➜  ~  pie run --stdout demo-custom-spec --mode baseline --delay 5
```

ingim@Workstation:~ ⌥⌘1

```
➜  ~ pie run --stdout demo-custom-spec --mode speculated --delay 5
```

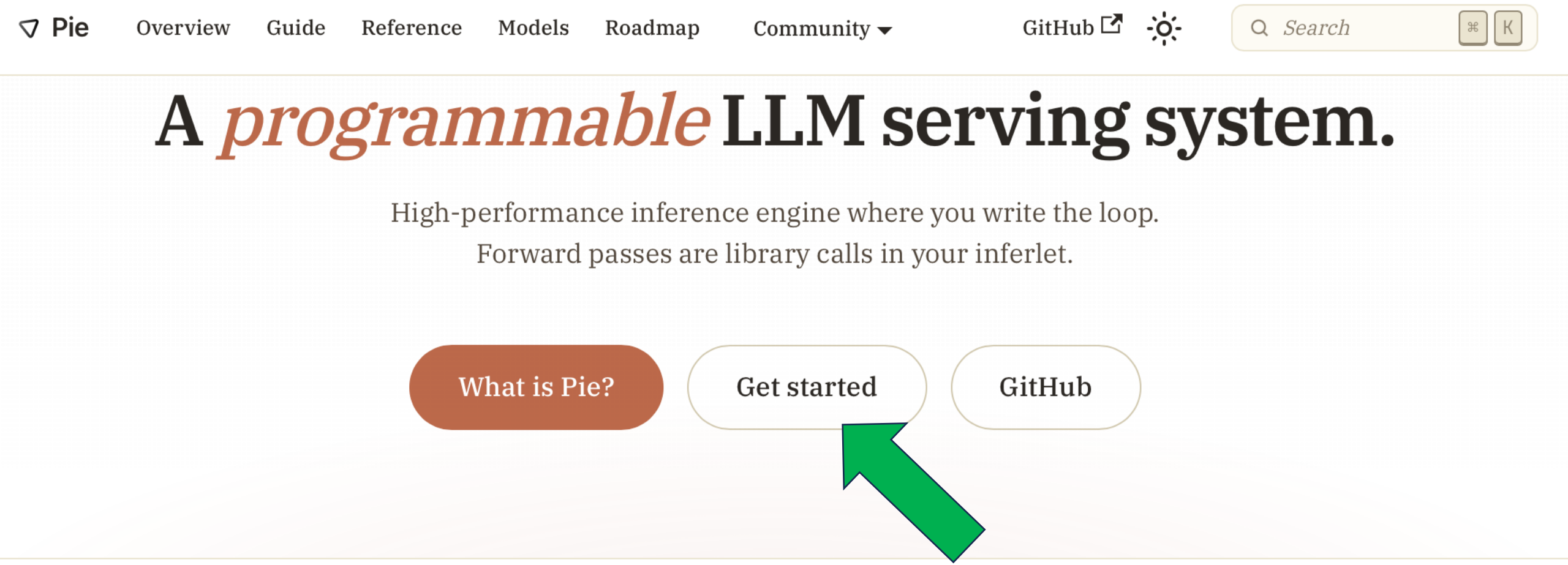
ingim@Workstation:~ ⌥⌘2

```
➜  ~ pie run --stdout demo-self-correct --mode baseline --delay 30
```

ingim@Workstation:~ ⌥⌘1

```
➜  ~ pie run --stdout demo-self-correct --mode verified --delay 30
```

Try it out: https://pie-project.org



Serve programs, not prompts

Try it out: https://pie-project.org

Linux and MacOS

```
curl -fsSL https://pie-project.org/install.sh | bash
pie config init
pie run text-completion -- --prompt "The capital of France is"
```

Join us to develop Pie further

- Open-source contributions
- Ph.D. student at Yale Computer Science

lin.zhong@yale.edu

- Engineer at Liszt AI

ceo@liszt.ai

SITE

pie-project.org

CODE

github.com/pie-project/pie

PAPER

SOSP'25
ingim.org/papers/gim2025pie.pdf

GOSIM

GOSIM Paris 2026

Thanks!